

LAB-07 명령어 Reference

명령어의 목적 · 예상 결과 · 오류 발생 시 확인 사항

각 명령어는 암기가 아니라 왜 실행하는지와 어떤 결과를 판단해야 하는지에 초점을 둡니다.

1. ros2 launch maintenance_lidar_demo lab_current.launch.py

- **명령어:** `ros2 launch maintenance_lidar_demo lab_current.launch.py`
- **목적:** 현재 `lab_active.yaml` 상태로 Mock LiDAR 와 Navigation Monitor 를 함께 실행합니다.
- **예상 결과:** `/lidar_node`, `/navigation_scan_monitor` 가 실행됩니다. Broken 상태에서는 `Laser scan data timeout` 경고가 발생합니다.
- **오류 발생 시 확인:** ROS 2 Humble source 여부, workspace build 여부, `source install/setup.bash` 여부를 확인합니다.

2. ros2 node list

- **명령어:** `ros2 node list`
- **목적:** 필요한 ROS 2 Node 가 실행 중인지 확인합니다.
- **예상 결과:** `/lidar_node`, `/navigation_scan_monitor` 가 표시됩니다.
- **오류 발생 시 확인:** `launch` 가 실행 중인지, 다른 터미널에서 ROS 환경을 source 했는지 확인합니다.

3. ros2 topic list

- **명령어:** `ros2 topic list`
- **목적:** 현재 ROS 2 graph 에 보이는 Topic 을 확인합니다.
- **예상 결과:** Broken v1.3 에서는 `/lidar/scan` 과 subscriber 가 만드는 `/scan` 을 관찰할 수 있습니다.
- **오류 발생 시 확인:** Node 실행 여부와 `ROS_DOMAIN_ID` 차이를 확인합니다.

4. ros2 topic info /scan -v

- **명령어:** `ros2 topic info /scan -v`
- **목적:** 기존 `/scan` 의 Publisher/Subscriber 연결 상태를 확인합니다.
- **예상 결과:** Broken 상태에서 `Publisher count: 0`, `Subscription count: 1` 이 핵심입니다.
- **오류 발생 시 확인:** `/scan` 이 `ros2 topic list` 에 존재하는지 먼저 확인합니다.

5. ros2 topic echo /lidar/scan --once

- **명령어:** `ros2 topic echo /lidar/scan --once`
- **목적:** 센서 데이터 자체가 실제 생성되고 있는지 확인합니다.
- **예상 결과:** `frame_id: laser_frame`, `ranges` 등의 `LaserScan` 메시지가 한 번 출력됩니다.
- **오류 발생 시 확인:** `/lidar/scan` Publisher 존재 여부와 `launch` 상태를 확인합니다.

6. ros2 topic hz /lidar/scan

- **명령어:** `ros2 topic hz /lidar/scan`
- **목적:** 현재 데이터 주기를 Baseline 과 비교합니다.
- **예상 결과:** 평균 약 10 Hz 가 관찰됩니다.

- 오류 발생 시 확인: 실제 메시지가 발행되는지 `ros2 topic echo ... --once` 로 먼저 확인합니다.

7. `git log --online --decorate --graph -5`

- 명령어: `git log --online --decorate --graph -5`
- 목적: 최근 변경과 v1.2, v1.3 태그 위치를 확인합니다.
- 예상 결과: baseline commit 과 LiDAR topic 변경 commit 을 구분할 수 있습니다.
- 오류 발생 시 확인: `prepare_git_history.sh` 실행 여부와 현재 경로가 LAB 저장소인지 확인합니다.

8. `git diff v1.2..v1.3`

- 명령어: `git diff v1.2..v1.3`
- 목적: 업그레이드 전후 변경 내용을 직접 비교합니다.
- 예상 결과: Publisher 의 scan_topic 이 /scan 에서 /lidar/scan 으로 변경된 차이가 나타납니다.
- 오류 발생 시 확인: `git tag --list` 로 두 태그가 존재하는지 확인합니다.

9. `./scripts/apply_topic_fix.sh`

- 명령어: `./scripts/apply_topic_fix.sh`
- 목적: 학생이 소스를 직접 편집하지 않고 Navigation Subscriber 를 /lidar/scan 으로 맞춥니다.
- 예상 결과: Applied v1.3 topic fix 메시지가 표시됩니다.
- 오류 발생 시 확인: 먼저 `./scripts/reset_to_v13.sh` 로 Broken 상태를 복원했는지 확인합니다.

10. `./scripts/rollback_v12.sh`

- 명령어: `./scripts/rollback_v12.sh`
- 목적: Fix 가 안전하지 않을 때 사용할 Baseline 복구 경로를 확인합니다.
- 예상 결과: Active configuration 이 v1.2 의 /scan → /scan 상태로 돌아갑니다.
- 오류 발생 시 확인: v1.2 태그 존재 여부를 확인합니다.